Fundament-Technik

QuizAway & Medien-Lern-App

Sync-Architektur · Datenpools · Fragenmodell · Entwicklerreferenz

Version 1.0 · März 2026 · Konsolidiertes Dokument (DOK-1)

Dieses Dokument beschreibt das technische Fundament beider Projekte: Sync-Architektur, Datenpools, Fragenmodell und Entwicklerreferenz.
Teil B (Datenpools) und Teil C (Fragenmodell) sind erste konkrete Instanziierungen des allgemeinen Modells am Beispiel Geo/QuizAway.
Andere Anwendungsfälle (Führerschein, Sprachenlernen, Chorübung) instanziieren dasselbe Modell mit anderen Pools und Schablonen.
Technologieentscheidungen und Datenstrukturen gelten für beide Projekte gleichermassen.

Teil A — Sync-Architektur

Beide Projekte teilen dieselben Grundbedürfnisse: lokale Datenpersistenz, Replikation zwischen Geräten und Spielern, und ein Gamification-System. Statt die Infrastruktur zweimal zu bauen, wird ein gemeinsames Modul entworfen das in beiden Projekten verwendet wird.

Kernentscheidung: Beide Projekte verwenden denselben Event-Ledger als Datenbasis.
Das Schema wird einmal designed und einmal implementiert.
Kein eigener Server wird betrieben — der Relay-Dienst ist Nostr.

A.1 Transportmodell — Ein Protokoll, fünf Zustände

Die Architektur kennt genau eine Protokollebene: IP. Wie eine IP-Verbindung zustande kommt — Mobilfunk, Heimnetz, Wi-Fi Direct zwischen zwei Geräten — interessiert die App nicht. Das Betriebssystem regelt das. Die App stellt nur eine einzige Frage: Ist ein Peer oder Relay erreichbar?

Zustand	Transport	Mechanismus	Wann
Immer	Lokal	IndexedDB + Event-Ledger	Kein Netz nötig — läuft immer
Erster Kontakt	Out-of-Band	SDP-Text per Messenger (WhatsApp, Threema ...)	Neuer Peer — oder Relay nicht erreichbar
Peers gleichzeitig online	WebRTC P2P direkt	WebRTC Data Channel, NIP-44	Bekannte Peers — Relay übernimmt Signaling
Nur Relay erreichbar	Nostr-Relay	WebSocket + NIP-44	Asynchron — Sender und Empfänger nicht gleichzeitig online
Keine Verbindung	Nur lokal	Ledger sammelt weiter	Sync beim nächsten Verbindungsaufbau

Kernprinzip: Lokale Persistenz läuft immer — die App funktioniert vollständig ohne Netzwerk.
Out-of-Band Signaling ist der bevorzugte erste Weg: kein Server, kein Account, nur ein Text per bestehendem Messenger.
WebRTC P2P wenn Peer bekannt und erreichbar. Nostr-Relay als Fallback wenn Peers nicht gleichzeitig online sind.

A.2 Schicht 1 — Lokale Persistenz (Event-Ledger)

Jede Nutzeraktion wird als unveränderlicher Eintrag in einem lokalen Event-Ledger gespeichert (IndexedDB). Niemals überschreiben — immer anhängen. Das ist das CRDT-Prinzip: da Events niemals geändert werden, gibt es keine Schreibkonflikte beim späteren Merge.

- Technologie: IndexedDB (funktioniert bei file://, kein Server nötig)
- Struktur: Append-Only-Log, jeder Eintrag hat eindeutige UUID
- Abgeleiteter Zustand (Punkte, Fortschritt) wird aus dem Ledger berechnet, nicht direkt gespeichert
- Beim Öffnen: Ledger laden, Zustand neu berechnen

> *Warum nicht localStorage? IndexedDB unterstützt grössere Datenmengen (>5 MB), strukturierte Objekte und asynchrone Zugriffe — notwendig für tausende Events über Monate.*

Event-Struktur

Feld	Typ	Beispiel	Bedeutung
id	UUID v4	a1b2-c3d4-...	Eindeutige Event-ID, unveränderlich
timestamp	ISO 8601 UTC	2026-03-15T09:00Z	Zeitstempel der Entstehung
playerId	UUID v4	f9e8-...	Gerätegenerierte Spieler-ID
type	String Enum	ANSWER_CORRECT	Art des Events
payload	JSON Object	{ questionId, score }	Event-spezifische Daten
schemaVersion	Integer	1	Für spätere Migrationen

Event-Typen

Event-Typ	Projekt	Payload-Felder
ANSWER_CORRECT	QuizAway, Lern-App	questionId, categoryId, durationMs
ANSWER_WRONG	QuizAway, Lern-App	questionId, categoryId, durationMs
DUEL_STARTED	QuizAway	opponentId, routeId, timestamp
DUEL_FINISHED	QuizAway	winnerId, scores: { p1, p2 }
REVIEW_SCHEDULED	Lern-App	itemId, nextReview, interval, ef
LEAGUE_JOINED	QuizAway, Lern-App	leagueId, playerName
XP_GAINED	QuizAway, Lern-App	amount, reason

Merge-Algorithmus

Der Merge zweier Ledger ist trivial, weil Events unveränderlich sind: Ledger A und Ledger B werden zusammengeführt als Vereinigung ihrer Event-Mengen. Duplikate werden per UUID erkannt und eliminiert. Der resultierende Zustand wird neu berechnet. Keine Konflikte, kein Koordinationsbedarf.

> *Vorwärtskompatibilität: Neue Event-Typen können jederzeit hinzugefügt werden. Ältere Clients ignorieren unbekannte Typen ohne Fehler.*

A.3 Transport A0 — Out-of-Band Signaling (Erster Kontakt)

Der bevorzugte erste Weg eine Verbindung herzustellen: kein Server, kein Account, keine Infrastruktur. Die App erzeugt ein SDP-Angebot (~500 Bytes Text), der Nutzer schickt es per

beliebigem Messenger an den anderen — der antwortet mit seiner SDP-Antwort. Danach steht die direkte WebRTC-Verbindung.

Weg	Wie	Am besten für
QR-Code scannen	App zeigt QR-Code, anderes Gerät scannt ihn	Beide im selben Raum
Text kopieren + senden	SDP-Text per Threema, Signal, WhatsApp, E-Mail	Beliebige Distanz, asynchron
Link teilen	SDP-Angebot als URL-Fragment kodiert — öffnet direkt die App	Komfortabelste Option
PIN via Rendezvous	Kurzlebiger lokaler Server tauscht SDP aus — 6-stelliger PIN	LAN-Testbetrieb (PoC AP6)

Das virale Prinzip: Jede neue Verbindung ist ein neuer Knoten im Netz.
Einmalig: ~2 Minuten Handshake per Messenger. Danach: App merkt sich den Peer — nächstes Mal automatisch.
Das SDP-Angebot ist mit dem Nostr-Privkey des Absenders signiert — kein Man-in-the-Middle möglich.

A.4 Transport A — WebRTC P2P (Bekannte Peers)

Wenn bekannte Peers gleichzeitig online sind, verbinden sie sich direkt über WebRTC Data Channels. Der Relay übernimmt nur noch das Signaling (Handshake) — danach läuft alles direkt. Kein Relay-Overhead, minimale Latenz.

Ablauf

- Peer-Discovery: App prüft ob bekannte Peers gerade erreichbar sind (via Nostr-Subscription)
- Handshake: SDP-Angebot und Antwort werden über den Relay ausgetauscht (NIP-44 verschlüsselt)
- Verbindung steht: ab jetzt direktes P2P über WebRTC Data Channel — Relay ist raus
- Ledger-Merge: beide tauschen ihre Deltas direkt aus, NIP-44 verschlüsselt
- Verbindung bleibt offen solange beide online — neue Events gehen direkt

STUN und TURN — Adressfindung über Netzgrenzen

WebRTC P2P funktioniert im lokalen Netz ohne Hilfe. Über Netzgrenzen (NAT, Firewalls) muss die öffentliche IP-Adresse gefunden werden. STUN löst das für die meisten Fälle — TURN ist der Fallback wenn NAT-Traversal scheitert.

Protokoll	Funktion	Wann nötig
STUN (Session Traversal Utilities for NAT)	Gerät erfährt seine öffentliche IP — ICE-Kandidaten werden gesammelt	Immer aktiv im Hintergrund
TURN (Traversal Using Relays around NAT)	Traffic wird über Relay-Server geleitet wenn direktes P2P scheitert	Symmetrisches NAT, strenge Firewall

ICE (Interactive Connectivity Establishment)	Testet alle möglichen Verbindungswege und wählt den besten	Automatisch bei jedem Verbindungsaufbau

Für LAN-Betrieb (PoC): kein STUN/TURN nötig — beide Geräte sind direkt erreichbar.
Für Produktionsbetrieb: öffentliche STUN-Server (stun.l.google.com:19302) reichen für ~85% der Fälle.
TURN nur wenn STUN scheitert — erhöht Latenz und verursacht Traffic-Kosten.

A.5 Transport B — Nostr-Relay (Asynchron, NIP-44)

Nostr ist ein offenes dezentrales Protokoll für verschlüsselte Nachrichten. Es dient als Relay wenn Peers nicht gleichzeitig online sind — und als Signaling-Helfer für den WebRTC-Handshake. Kein eigener Server nötig: öffentliche Nostr-Relays laufen rund um die Uhr, kostenlos, ohne Registrierung.

Verwendete Nostr-Standards (NIPs)

NIP	Funktion	Verwendung im Projekt
NIP-01 — Basis-Protokoll	Events, Subscriptions, Filter	Grundlage aller Nostr-Kommunikation
NIP-04 — Encrypted DMs (veraltet)	Verschlüsselte Direktnachrichten (AES-CBC)	Nur als Fallback — NIP-44 bevorzugt
NIP-44 — Encrypted Payloads	Verschlüsselte Inhalte (ChaCha20-Poly1305)	Standard für alle Projekt-Nachrichten
NIP-57 — Lightning Zaps	Kryptographischer Zahlungsnachweis	Payment-Integration (Phase 2)

NIP-44 Verschlüsselung

NIP-44 verwendet ECDH (Elliptic Curve Diffie-Hellman) für den Schlüsselaustausch und ChaCha20-Poly1305 für die symmetrische Verschlüsselung. Jede Nachricht ist für genau einen Empfänger verschlüsselt — der Relay sieht nur Chiffretext.

- Schlüsselpaar: secp256k1 — identisch mit Bitcoin/Nostr, erzeugt beim ersten App-Start
- ECDH: gemeinsames Geheimnis aus eigenem Privkey + Pubkey des Gegenübers
- ChaCha20-Poly1305: symmetrische Verschlüsselung + Authentizitätsprüfung
- Jede Nachricht hat eine neue zufällige Nonce — kein IV-Wiederholungsproblem

Öffentliche Relay-Server

Relay	URL	Bemerkung
Damus	wss://relay.damus.io	Grosser öffentlicher Relay, gut verfügbar
nos.lol	wss://nos.lol	Zuverlässig, geringer Spam
Nostr.Band	wss://relay.nostr.band	Gute Suchbarkeit, für Discovery

Teil B — Datenpools: Geo-Wissensbasis

Dieser Teil beschreibt die Geo-Wissensbasis als ersten konkreten Datenpool — die erste Instanziierung des allgemeinen Pool-Modells.
Das Schema ist generisch: andere Anwendungsfälle (Führerschein, Sprachenlernen, Chorübung) instanziieren dasselbe Modell mit anderen Pools.
Was hier für 'staedte', 'bundeslaender' und 'fluesse' beschrieben wird, gilt strukturell für jeden Wissenspool.

Leitprinzip: Deutschland ist ein Attribut, kein Scope. Jeder Pool ist global definiert — DE-spezifische Einträge sind vollständiger, aber das Schema schliesst keine andere Region aus.

B.1 Pool-Übersicht

Pool	Objekte (ca.)	Primärquelle	Bemerkung
staat	250	Wikidata / ISO 3166	Basis für alle anderen Pools
kontinent	7	Wikidata	Attribut von staat
bundesland	16 (DE)	Wikidata / Destatis	Untereinheit von staat — nur DE vollständig
kreis	400 (DE)	Destatis / Wikidata	Untereinheit von bundesland
stadt	~12.000	GeoNames / Destatis	Kernpool · selbstreferenzierend für Stadtteile
kfz_kennzeichen	~500 (DE)	KBA	1:1 mit kreis verknüpft
plz	~8.200 (DE)	Deutsche Post / OSM	Polygon → räumliche Abfragen
vorwahl	~5.200 (DE)	Bundesnetzagentur	Verknüpft mit ort / kreis
autobahn	~60 (DE)	OSM / BASt	Länge, Bundesländer, Anschlussstellen
bahnstrecke	~300 (DE)	DB Open Data / OSM	ICE/IC/RE-Linien · Halte als Relation zu bahnhof
bahnhof	~5.700 (DE)	DB Open Data / OSM	FK zu stadt · Kategorie 1–7 (DB-Schema)
fluss	~4.000 (DE)	HydroRIVERS / GeoNames	muendet_in → fluss (Nebenfluss als Relation)
see	~1.500 (DE)	GeoNames / OSM	Ableitbar: Städte am See
meer	~20	Natural Earth / Wikidata	Attribut von staat / küstenlänge
berg	~3.000 (DE)	GeoNames (P/MT)	Gebirge als Attribut / Gruppe
insel	~200 (DE)	GeoNames (ISL)	Zugehörigkeit zu staat / bundesland

B.2 Geografisch-politische Pools

Pool: staat

Fundamentaler Basispool. Alle anderen Pools referenzieren staat direkt oder indirekt. Deutschland ist ein Eintrag wie jeder andere — kein Sonderfall im Schema.

Attribut	Typ	Quelle	Bemerkung
id	TEXT PK	ISO 3166-1 alpha-2	z.B. 'DE', 'FR', 'PL'
kontinent_id	TEXT FK	Wikidata (P30)	→ Pool kontinent
hauptstadt_id	TEXT FK	Wikidata (P36)	→ Pool stadt
flaeche_km2	REAL	Wikidata (P2046)	Gesamtfläche in km²
einwohner	INTEGER	Wikidata (P1082)	Aktuellster verfügbarer Wert
lat / lon	REAL	Wikidata (P625)	Zentroid-Koordinate
kfz_code	TEXT	Wikidata (P395)	Internationales KFZ-Kennzeichen: 'D'
tld	TEXT	Wikidata (P78)	Top-Level-Domain: '.de'
un_mitglied	INTEGER	Wikidata	0/1 — Boolean
eu_mitglied	INTEGER	Wikidata	0/1 — Boolean
— Namen —	→ translations	Kap. B.5	name, name_nativ · je Sprache eine Zeile

Pool: bundesland

Untereinheit von staat. Vollständig für Deutschland (16 Einträge), schema-kompatibel für andere Föderalstaaten (US-States, österreichische Bundesländer, Schweizer Kantone).

Attribut	Typ	Quelle	Bemerkung
id	TEXT PK	ISO 3166-2	z.B. 'DE-BY', 'DE-NW'
staat_id	TEXT FK	ISO 3166-1	→ Pool staat
hauptstadt_id	TEXT FK	Wikidata	→ Pool stadt
flaeche_km2	REAL	Destatis	Fläche in km²
einwohner	INTEGER	Destatis	Aktuellster Wert
einwohner_dichte	REAL	ableitbar	einwohner / flaeche_km2
bip_mrd_eur	REAL	Statistikamt	Bruttoinlandsprodukt
gruendungsjahr	INTEGER	Wikidata	Jahr der aktuellen Gründung
lat / lon	REAL	Wikidata / OSM	Zentroid
— Namen —	→ translations	Kap. B.5	name · je Sprache eine Zeile

Pool: stadt

Kernpool. Enthält alle Städte, Gemeinden und — selbstreferenzierend — Stadtteile. Die Ebene wird über das Attribut 'ebene' und 'parent_id' gesteuert.

Attribut	Typ	Quelle	Bemerkung
id	TEXT PK	GeoNames ID	Eindeutig global

staat_id	TEXT FK	ISO 3166-1	→ Pool staat
bundesland_id	TEXT FK	ISO 3166-2	→ Pool bundesland (nullable)
kreis_id	TEXT FK	Destatis	→ Pool kreis (nullable)
parent_id	TEXT FK	selbstreferenz.	→ stadt — für Stadtteile
ebene	TEXT	ableitbar	'stadt' | 'stadtteil' | 'ortsteil'
lat / lon	REAL	GeoNames	Koordinate
einwohner	INTEGER	GeoNames / Destatis	Nullable
hoehe_m	INTEGER	GeoNames / SRTM	Höhe über NN
ist_kuestenstadt	INTEGER	OSM-Pipeline	0/1 — aus Geometrie abgeleitet
— Namen —	→ translations	Kap. B.5	name · je Sprache eine Zeile

Stadtteil-Entscheidung: Germering ist kein Stadtteil von München — die Frage 'In welchem Stadtteil liegt Germering?' wird korrekt nicht generiert, weil parent_id = null.

B.3 Infrastruktur-Pools (DE-spezifisch)

Pool: kfz_kennzeichen

Attribut	Bemerkung
id TEXT PK	z.B. 'M', 'B', 'HH'
kreis_id TEXT FK	→ Pool kreis
aktiv INTEGER	0/1 — historische Kennzeichen als inaktiv markiert
vergabe_seit INTEGER	Jahr der ersten Vergabe

Pool: bahnhof

Attribut	Bemerkung
id TEXT PK	DB-interne Stations-ID
stadt_id TEXT FK	→ Pool stadt
kategorie INTEGER	1–7 nach DB-Schema (1 = Hauptbahnhof, 7 = Haltepunkt)
lat / lon REAL	Koordinate aus GTFS
gtfs_stop_id TEXT	Stop-ID aus GTFS-Feed — für Strecken-Relation
— Namen — → translations	name · je Sprache

Pool: bahnstrecke

Attribut	Bemerkung
id TEXT PK	DB-Route-ID aus GTFS
typ TEXT	'ICE' | 'IC' | 'RE' | 'S'
laenge_km REAL	Streckenlänge in km (ableitbar aus Haltefolge + Koordinaten)
— Namen — → translations	Linienbezeichnung z.B. 'ICE 25 Hamburg–München'

Strecke–Bahnhof-Relation: separate Tabelle strecke_halte mit (strecke_id, bahnhof_id, reihenfolge, distanz_km_ab_start).

B.4 Natürliche Pools

Pool: fluss

Attribut	Bemerkung
id TEXT PK	HydroRIVERS-ID
muendet_in_id TEXT FK	→ fluss (selbstreferenz. für Nebenflüsse)
laenge_km REAL	Gesamtlänge
staat_id TEXT FK	Hauptstaat (längster Abschnitt)
— Namen — → translations	name · je Sprache

Relation stadt_fluss: separate Tabelle (stadt_id, fluss_id) — erzeugt durch 5-km-Puffer-Join aus HydroRIVERS-Polylinien.

Pools: see, berg, meer, insel

Pool	Schlüsselattribute	Abgeleitete Relationen
see	id, staat_id, flaeche_km2, max_tiefe_m	stadt_see (3-km-Puffer-Join)
berg	id, bundesland_id, hoehe_m, gebirge_gruppe	—
meer	id, typ ('Ozean'|'Meer'|'See')	staat_meer (Küstenlänge als Relation-Attribut)
insel	id, staat_id, bundesland_id, flaeche_km2	—

B.5 Translations-Tabelle & Pipelines

Translations-Tabelle

Alle mehrsprachigen Namen landen in einer zentralen translations-Tabelle. Kein Pool enthält direkte name-Spalten — alle Bezeichnungen werden über translations aufgelöst.

Spalte	Typ	Beispiel	Bedeutung
pool	TEXT	'staat'	Welcher Pool
objekt_id	TEXT	'DE'	ID des Objekts im Pool
key	TEXT	'name'	name | name_ascii | name_nativ | alternat_N
lang	TEXT	'de'	ISO 639-1 Sprachcode
value	TEXT	'Deutschland'	Der Wert

Primärschlüssel: (pool, objekt_id, key, lang) — eindeutig.
Fallback: jeder Eintrag hat mindestens lang='de'. Fehlt lang='en', wird 'de' zurückgegeben.
Schlüssel: 'name' (Standardname), 'name_ascii' (ohne Sonderzeichen), 'name_nativ' (Eigenbezeichnung), 'alternat_N' (historische Namen).

Build-Pipelines

Pipeline	Quellen	Erzeugt
AP1 — Basispools	GeoNames allCountries.zip, Destatis-CSV	staat, bundesland, kreis, stadt (ebene='stadt')
AP2 — Geometrie	HydroRIVERS, OSM Overpass, Natural Earth, SRTM, Shapely	stadt_fluss, stadt_see, ist_kuestenstadt, berg.bundesland_id
AP3 — DE-spezifisch	KBA-Datensatz, Deutsche Post PLZ, Bundesnetzagentur, DB GTFS	kfz_kennzeichen, plz, vorwahl, bahnstrecke, bahnhof, strecke_halte
AP4 — Translations	GeoNames alternateNames.zip, Wikidata SPARQL	translations (name_de, name_en, Altersnamen für alle Pools)

Teil C — Fragenmodell

Dieser Teil beschreibt das Fragenmodell anhand von Geo-Beispielen (QuizAway).
Das Modell ist generisch: Fragetypen, Seeds und Distraktor-Strategien gelten für jeden Wissenspool.
Andere Anwendungsfälle instanziieren dasselbe Modell: Führerschein-Fragen als Katalog-Typ, Sprachvokabeln als attributabfrage-Typ.

C.1 Zwei Fragentypen — eine Schnittstelle

Das System kennt zwei grundlegend verschiedene Arten von Fragen. Sie entstehen auf unterschiedlichen Wegen, aber die App unterscheidet sie nicht — sie bekommt in beiden Fällen dasselbe Objekt zur Anzeige.

Kriterium	Typ 1 — Generativ	Typ 2 — Katalog
Herkunft	Normalisierte Datenpools	Amtlicher / redaktioneller Datensatz
Generierung	Laufzeit (Browser, JS)	Build-Zeit (Pipeline oder Import)
Gespeichert als	Frage-Seed + Pool-Referenz	Vollständige Frage als Text
Versionierung	Pool-Schema-Version im Seed	Katalog-Version im Datensatz
Neue Inhalte	Automatisch bei neuen Daten	Manueller Import neuer Katalogversion
Mehrsprachig	Schablone je Sprache	Übersetzung je Katalog
Beispiele	Geografie, Bahn, KFZ, Höhe ...	Führerschein B, Jägerprüfung ...

Laufzeit-Schnittstelle: Beide Typen liefern { fragetext, antworten[4], richtig_index, erklaerung, typ, quelle_id }. Die App fragt nie nach der Herkunft.

C.2 Generative Fragen — das Seed-Modell

Eine generative Frage wird nicht als Text gespeichert, sondern als Seed — eine kompakte Beschreibung wie die Frage erzeugt werden soll. Der Text entsteht erst zur Laufzeit aus Seed + aktuellen Pooldaten + Sprachschablone.

Struktur eines Frage-Seeds

```
{ "id": "seed_q0001",
  "pool": "stadt",
  "pool_schema_v": 1,
  "objekt_id": "augsburg",
  "fragetyp": "attributabfrage",
  "attribut": "bundesland_id",
  "schwierigkeit": "L",
  "distraktor_strat": "gleiche_region",
```

```
    "lang": "de" }
```

Aus diesem Seed erzeugt der Generator zur Laufzeit: Fragetext: "In welchem Bundesland liegt Augsburg?" — Richtige Antwort: "Bayern" — Drei Distraktoren nach Strategie — Erklärung: "Augsburg liegt in Bayern — Landeshauptstadt ist München."

C.3 Fragetypen

Jeder Fragetyp ist eine Schablone die die Struktur einer Frage beschreibt — unabhängig vom Inhalt. Er deklariert welche Attribute und Pools er braucht und was die richtige Antwort ist. Die Fragetypen sind pool-agnostisch — sie funktionieren mit jedem Pool der die nötigen Attribute hat.

Fragetyp	Schablone	Braucht	Geo-Beispiel
attributabfrage	Was ist [Attribut] von [Objekt]?	1 Pool, 1 Attribut	Bundesland einer Stadt
identifikation	Welches Objekt hat [Attribut] = [Wert]?	1 Pool, 1 Attribut	Welche Stadt hat Kennzeichen 'M'?
vergleich	Welches hat den grössten/kleinsten [Attribut]?	1 Pool, 1 Zahl-Attribut	Höchste Einwohnerdichte
zuordnung	Zu welchem [Pool B] gehört [Objekt aus A]?	2 Pools, 1 Verknüpfung	KFZ-Kennzeichen → Stadt
nachbar	Was grenzt an X?	1 Pool, Relation grenzt_an	Nachbarland von Deutschland
rangfolge	Was kommt nach X auf Route Y?	1 Pool, geordnete Sequenz	Nächster ICE-Halt
schaetzung	Wie weit / hoch / gross ist X?	1 Pool, numerisches Attribut	Distanz zur Grenze
epoche	Aus welcher Zeit stammt X?	1 Pool, gruendungsjahr	Gründungsepoche einer Stadt

C.4 Distraktor-Strategien

Die Qualität einer Frage hängt massgeblich von den Falschantworten ab. Die Strategie steuert wie ähnlich die Falschantworten zur richtigen Antwort sein sollen. Sie ist pool-agnostisch — 'gleiche_region' bedeutet beim Geo-Pool geografische Nähe, beim Führerschein-Pool thematische Nähe.

Strategie	Geo-Beispiel	Allgemeine Bedeutung
gleicher_pool	3 andere Bundesländer	Zufällig aus demselben Pool — Basisstrategie, immer verfügbar
gleiche_region	Nachbar-Bundesländer	Aus benachbarten / verwandten Einträgen — schwerer
aehnlicher_wert	Städte mit ähnlicher Einwohnerzahl	Numerisch ähnliche Werte — für Schätzfragen
gleiche_kategorie	Andere Grossstädte > 500.000 EW	Gleicher Übertyp / gleiche Kategorie
historisch	Frühere Kennzeichen	Plausibel aber falsch — für historische Fragetypen

C.5 Fragenkataloge — Typ 2

Amtliche oder redaktionell erstellte Fragenkataloge werden als vollständige Texte importiert. Sie unterliegen keiner generativen Logik — ihr Inhalt ist normativ vorgegeben.

Katalog-Metadaten

Feld	Typ	Beispiel	Bedeutung
id	TEXT PK	'fuehrerschein_b_2024'	Katalog-ID mit Version im Namen
version	TEXT	'2024-01'	Ausgabedatum des Herausgebers
herausgeber	TEXT	'BMDV'	Bundesministerium für Digitales und Verkehr
gueltig_ab	TEXT	'2024-01-01'	ISO-Datum
gueltig_bis	TEXT	null	null = aktuell gültig
sprache	TEXT	'de'	Sprache des Katalogs
fragen_anzahl	INTEGER	1080	Gesamtzahl der Fragen

C.6 Versionierung & Duel-Reproduzierbarkeit

Im Duel-Modus müssen beide Geräte exakt dieselbe Frage erzeugen — ohne vorherige Übertragung des Fragetextes. Das gelingt über deterministischen Seed + identische Pooldaten auf beiden Geräten.

Duel-Reproduzierbarkeit

- Beide Geräte haben dieselbe SQLite-Datenbank (aus Build-Pipeline)
- Seed wird deterministisch ausgewählt: random_seed im Duel-Event bestimmt die Fragenfolge
- Generator läuft auf beiden Geräten lokal — identische Eingabe → identische Ausgabe
- Über den DataChannel wird nur der Antwort-Index übertragen (0–3), nicht der Fragetext

Pool-Versionierung

Versions-Typ	Bedeutung	Konsequenz für Seeds
pool_schema_v (Breaking Change)	Attribut umbenannt, Typ geändert, Pool neu strukturiert	Seed-Status → review_needed, manuelle Prüfung nötig
objekt_version (inhaltlich)	Einwohnerzahl aktualisiert, Name korrigiert	Seed bleibt gültig — Generator erzeugt aktualisierten Text
Seed-Status: active	Seed funktioniert mit aktueller Pool-Version	Wird normal ausgespielt
Seed-Status: review_needed	Pool-Schema hat sich verändert	Nicht ausgespielt bis manuell geprüft
Seed-Status: invalid	Objekt existiert nicht mehr im Pool	Dauerhaft deaktiviert

Teil D — Entwicklerreferenz

D.1 Datenstrukturen

Event-Ledger Entry

```
interface LedgerEvent {
  id:            string;   // UUID v4
  timestamp:     string;   // ISO 8601 UTC
  playerId:      string;   // UUID v4, gerätegeneriert
  type:          EventType;
  payload:       Record<string, unknown>;
  schemaVersion: number;   // aktuell: 1
}
```

Frage-Seed

```
interface FrageSeed {
  id:              string;   // 'seed_q0001'
  pool:            string;   // 'stadt' | 'bundesland' | ...
  pool_schema_v:   number;
  objekt_id:       string;
  fragetyp:        FrageTyp;
  attribut?:       string;   // für attributabfrage
  schwierigkeit:   'L'|'M'|'S';
  distraktor_strat: DistraktorStrategie;
  lang:            string;   // ISO 639-1
  status:          'active'|'review_needed'|'invalid';
}
```

Laufzeit-Frage (gemeinsame Schnittstelle)

```
interface LaufzeitFrage {
  fragetext:    string;
  antworten:    [string, string, string, string];
  richtig_index: 0|1|2|3;
  erklaerung:   string;
  typ:          'generativ'|'katalog';
  quelle_id:    string;   // seed_id oder katalog_frage_id
}
```

Nostr-Keypair

```
interface NostrKeypair {
  privKey: string;  // 32 Bytes hex — secp256k1
  pubKey:  string;  // 32 Bytes hex — x-Koordinate des Public Key
}
```

D.2 JS-Module & Schnittstellen

Modul	Exportierte Funktionen	Abhängigkeiten
EventLedger	append(event), getAll(), merge(other),	IndexedDB

	exportDelta(since)	
FrageGenerator	generateFrage(seed, poolDB, lang), generateDistraktoren(seed, poolDB)	SQLite (sql.js)
NostrClient	connect(relays), sendDM(recipientPub, payload), subscribe(filter)	WebSocket, secp256k1
WebRTCPeer	createOffer(), processOffer(sdp), applyAnswer(sdp)	RTCPeerConnection
RendezvousClient	getPin(), postOffer(pin, sdp), pollAnswer(pin)	fetch() — nur für PoC/LAN

D.3 SQLite Build-Pipeline

Die Geo-Wissensbasis wird als SQLite-Datenbank gebaut und als Base64 in die App eingebettet oder als separater Download bereitgestellt. Die Pipeline läuft lokal am PC — kein Cloud-Dienst nötig.

Schritt	Script / Tool	Output
1. GeoNames laden	Python: download_geonames.py	allCountries.txt (ca. 50 MB)
2. Basispools befüllen	Python: build_pools.py	geo.sqlite (staat, bundesland, kreis, stadt)
3. Geometrie-Joins	Python: build_geometry.py (Shapely)	Relationen: stadt_fluss, stadt_see, ist_kuestenstadt
4. DE-spezifisch	Python: build_de_pools.py	kfz_kennzeichen, plz, vorwahl, bahnhof, bahnstrecke
5. Translations	Python: build_translations.py	translations-Tabelle für alle Pools
6. Seeds generieren	Python: generate_seeds.py	seeds.json (ca. 50.000 Einträge)
7. In App injizieren	Python: inject_db.py	quizaway_v2.html mit eingebetteter DB

Testkriterium AP1: SELECT COUNT(*) FROM stadt WHERE staat_id='DE' → ca. 12.000 Einträge
*Testkriterium AP3: SELECT * FROM kfz_kennzeichen WHERE id='M' → München, aktiv=1*
Testkriterium AP4: SELECT value FROM translations WHERE pool='staat' AND objekt_id='DE' AND key='name' AND lang='en' → 'Germany'

D.4 Technologie-Stack Übersicht

Bereich	Technologie	Begründung
Lokale DB	SQLite via sql.js (WASM)	Vollständig im Browser, kein Server
Persistenz	IndexedDB	>5 MB, strukturiert, asynchron
Kryptographie	secp256k1 via noble-secp256k1	Nostr-kompatibel, reine JS-Lib
Verschlüsselung	AES-256-GCM (WebCrypto) / ChaCha20-Poly1305	WebCrypto nativ im Browser
P2P	WebRTC Data Channel	Browser-nativ, kein Plugin

Relay	Nostr (WebSocket)	Öffentlich, kostenlos, kein eigener Server
Frontend QuizAway	Vanilla JS, standalone HTML	Keine Dependencies, E-Mail-tauglich
Frontend Lern-App	React + TypeScript + Vite	PWA, Typsicherheit, OSMD-kompatibel
Build-Pipeline	Python 3.11	GeoNames, Shapely, GTFS-Parser